



re-forge

The *compounding layer*
for your AI toolchain.

Akasxh & Shine · UCWS Singapore 2026

● #1 on the leaderboard

● 2 papers @ ICML 2026

● open-source · model-agnostic

✦ re-forge.vercel.app



re-forge.vercel.app



github.com/Akasxh/re-forge

■ THE HOOK

Every session starts from *zero*.

You run dozens of AI coding sessions a day — Claude Code, Cursor, Codex, ChatGPT. They forget what worked, repeat your mistakes, and leave no trace.

You are the only memory in the loop. That doesn't scale, and it doesn't compound.

0

memory carried between sessions — every tool, every day

1

thing that remembers what worked yesterday — **you**

THE PROBLEM

A single agent is confident,
and *unaccountable*.

01 It hallucinates, anchors on its first idea, and forgets yesterday.

02 On long-horizon work that's wasted tokens, repeated bugs, and **no audit trail**.

You can't prove what it checked.

03 The most valuable thing — **how your team actually works** — evaporates.

Gone at the end of every session.

■ THE INSIGHT

The edge isn't a smarter model.

Everyone rents the same models. Renting a bigger one buys nobody a moat.

The edge is the *operating layer* on top of them.

- coordination
- adversarial verification
- evidence on disk
- memory that compounds

WHAT RE-FORGE IS

One prompt → a whole *org.*

A coordinated org of specialist agents — research, engineering, security, testing, docs — plus a Forge meta-agent. Model-agnostic: it's a procedure, not a model.

- ❑ **Specialists fan out** from one lead — each lane works in parallel.
- ❑ **Skeptic + adversary attack** the work — weak claims die before you see them.
- **Survivors reach SYNTHESIS** — evidence lands on disk, memory gains a lesson.



PROOF — RECEIPTS, ALL REAL AND CLICKABLE

Proven on the hardest user there is: *itself*.

01

Compute-Aware Mixture-of-Agents: Verifier-Gated Adaptive Aggregation under a Fixed Token Budget

Shine Gupta · S Akash — SCALE Workshop, ICML 2026

public at camera-ready

02

Who Hallucinates Tools, How Often, and What Fixes It?

S Akash · Shine Gupta — SCALE Workshop, ICML 2026

public at camera-ready

#1 / 81 votes

most of any project on the hackathon
leaderboard

685 checks

machine checks — green CI on every push

128 skills · 6

teams

it tests itself the way it tests your code

THE DEMO • SECURE JWT AUTH

● PLAIN AGENT • GENUINELY GOOD

A plain agent built JWT auth — and did it well.

19 / 19 its own security tests passed

The token core is sound — Argon2id, refresh rotation, reuse-detection, revocation. re-forge's report says so. We give credit.

By every signal the agent gives you, this looks shippable.

● RE-FORGE SECURITY TEAM • VERIFIED

One **HIGH** hole the 19 tests missed:

- ✗ **Broken object-level authorization (IDOR)** — any logged-in user can **read and DELETE** every other user's data.

reproduced cross-user • against the running app

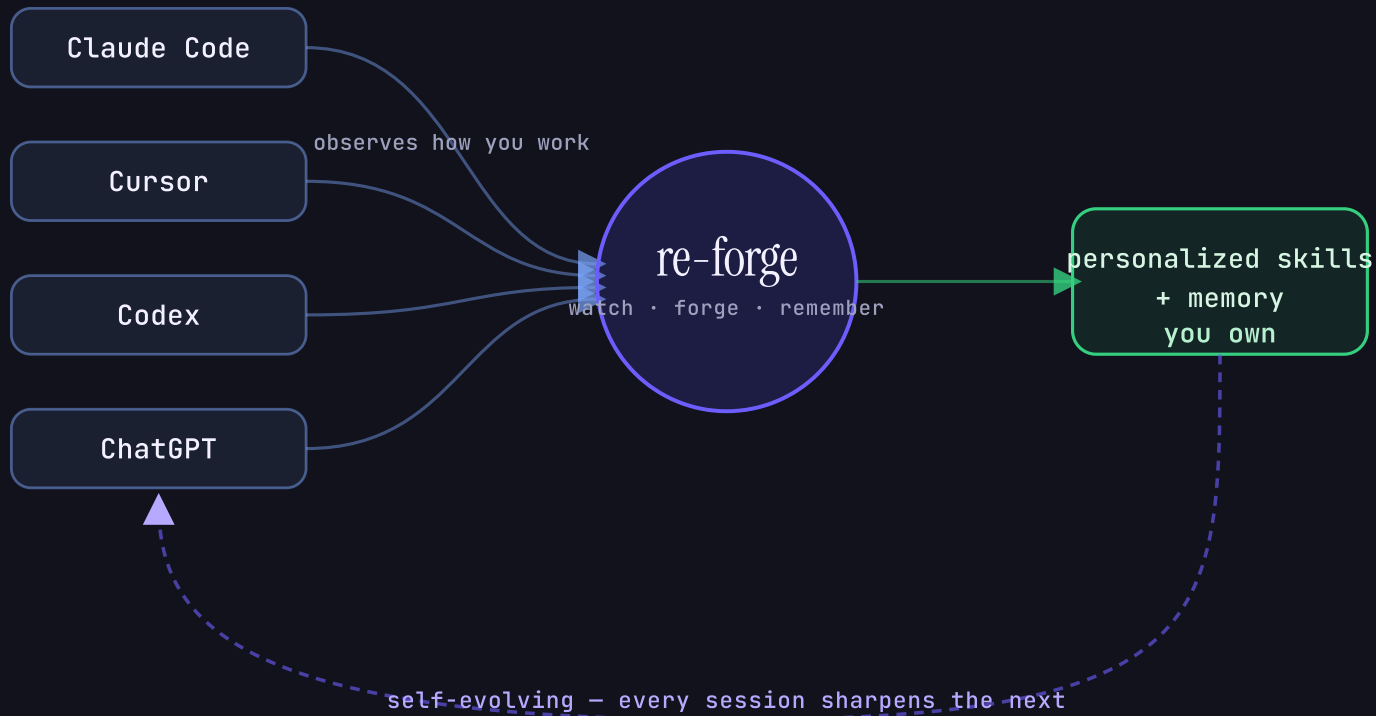
+ MEDIUM GAPS

- ✗ **No rate limiting** — brute-force wide open
- ✗ **Username enumeration** — login reveals who exists

adversary refused to inflate — it doesn't cry wolf

verdict: ADVISORY • 1 HIGH + 5 MEDIUM

Green tests are not secure. *That gap — "looks done" vs "verified" — is the product.*



THE PRODUCT

✓ ships today for Claude Code

A memory you *own*.

Today, re-forge already watches your Claude Code sessions and forges skills from them. **The company makes that always-on and cross-tool.**

It sits quietly across your tools, observes how your team actually works, and forges personalized, self-evolving skills & plugins — automatically.

roadmap: always-on cross-tool monitor

■ WHY IT COMPOUNDS

Every session makes it *stickier*.

The longer a team uses it, the more it knows how they work. Your layer gets better with use — the opposite of a tool you outgrow.

- 01 That personal / team memory is **your data, your patterns**.
- 02 No base model and no competitor can replicate it.
- 03 **A real network effect, per team.**

■ MARKET & WHY NOW

2026: agents are being handed *real engineering work.*

THE MISSING LAYER

Trust, audit, and memory — not another model.

OUR POSITION

Model-agnostic by design — we win no matter which model wins.

THE WEDGE

AI-heavy eng teams & research **groups** drowning in disconnected sessions.

honest status: open-source today, pre-revenue — the cross-tool layer is what we're building

■ BUSINESS MODEL — OPEN-CORE

- FREE / OSS

The funnel & the proof.

- The **operating procedure** — teams of specialist agents.
- The **skill library** — open, inspectable, model-agnostic.
- Every run is public proof the layer works.

where we monetize

- PAID · PER-SEAT SAAS

The always-on layer.

- The **always-on cross-tool monitor**.
- **Team-shared memory** + personalized skill-forging.
- An **audit / mission-control dashboard**.

Land with **one developer's** personal layer; expand to the **team's shared brain**.

■ TEAM & THE ASK

We shipped two ICMML papers *by using the product.*

Akasxh & Shine — we are our own first power users.

THE ASK **Design partners** — AI-heavy engineering teams.

- + **Intros** to AI-heavy eng & research orgs.
- + **Pre-seed** to build the hosted cross-tool layer.

Try it now →



re-forge.vercel.app



github.com/Akasxh/re-forge

■ APPENDIX • THE PRODUCT • RE-FORGE CLOUD

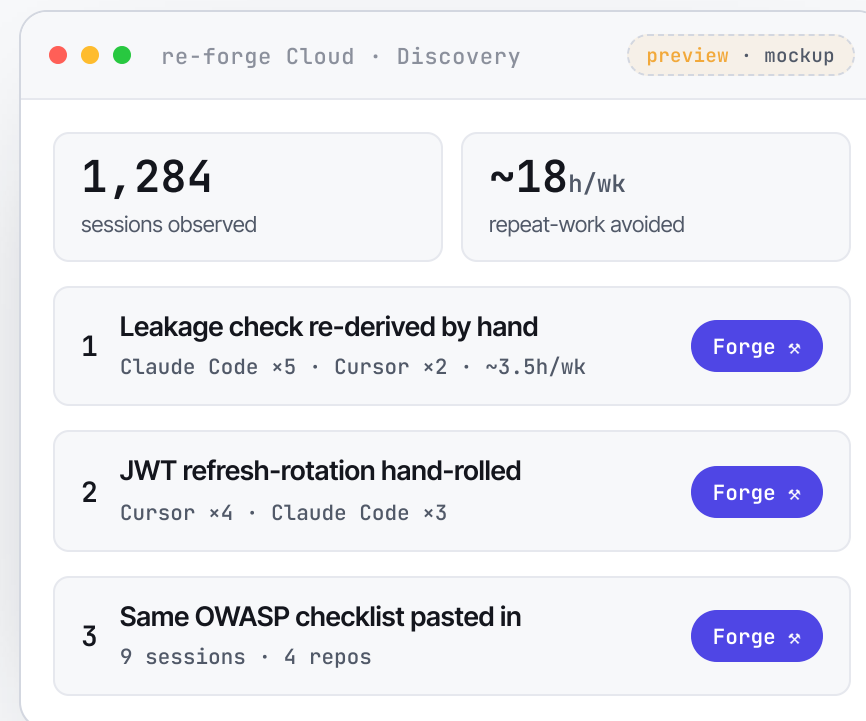
AutoStep for your AI *workforce*.

The bottleneck isn't the model — it's *discovery*. No one can see how their agents actually work, so improving them is guesswork.

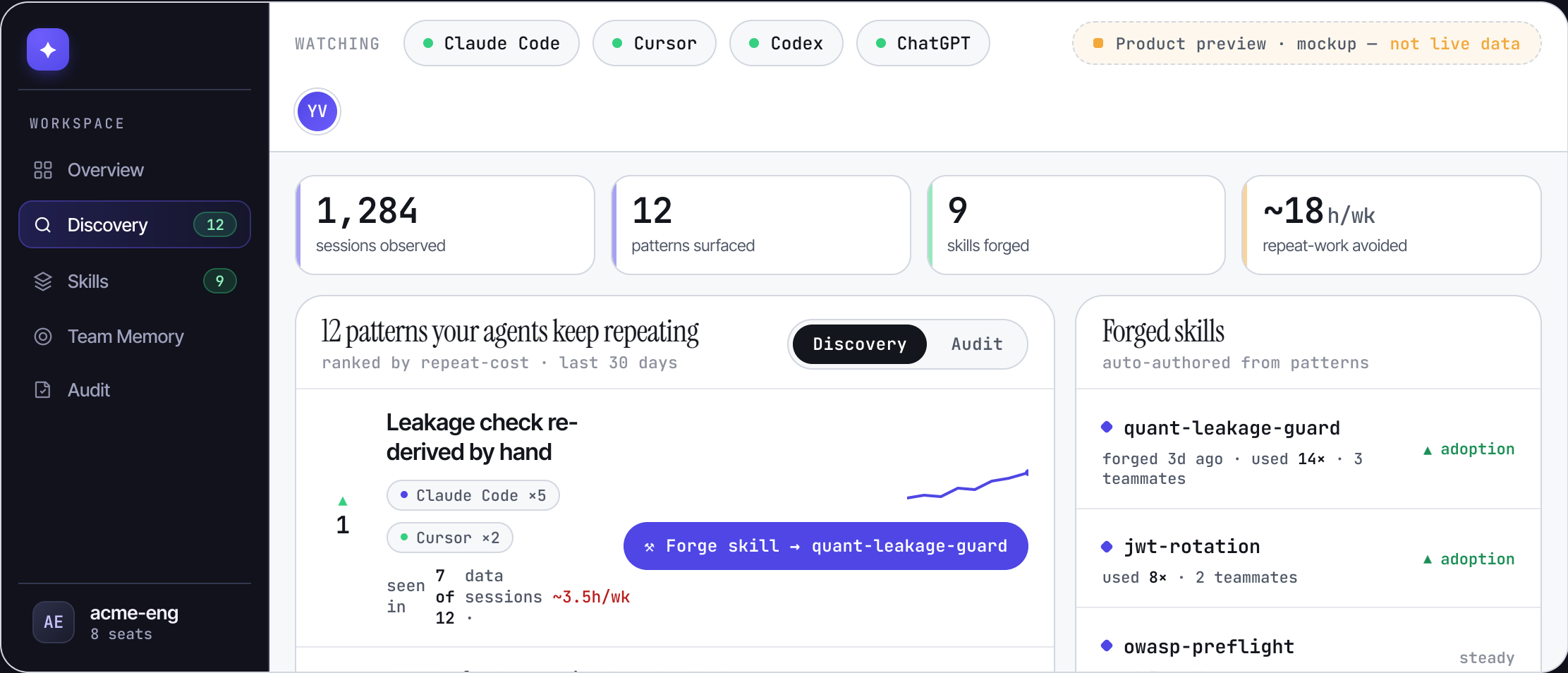
re-forge Cloud watches your agents — **not your people** — across every tool, finds what they repeat and what they get wrong, and **forges the skills that fix it**.

Self-evolving. The shared brain your whole team's agents draw from.

■ Product preview • mockup — not live data



APPENDIX • THE PRODUCT • LIVE MOCK



It finds the work your agents repeat — and forges the skill that ends it.

re-forge

reforge.vercel.app • appendix

APPENDIX · THE PRODUCT · THE MOAT THAT COMPOUNDS

Forged skills

adoption climbs with use

◆ quant-leakage-guard

forged 3d ago · used 14× · 3 teammates

▲ adoption

◆ jwt-rotation

used 8× · 2 teammates

▲ adoption

◆ owasp-preflight

used 5× · 1 teammate

steady

Team memory

the shared brain · the moat

47 lessons · shared across 8 seats

- purge + embargo before any time-series CV

used 31× · top lesson

- evidence-first: every claim links its source

used 26×

Every session sharpens the shared brain. Every claim leaves a trail.

Audit

every claim leaves a trail you can open

#4127

Claude Code
acme/api

● SECURITY GATE · KILLED

killed claim *"tokens are revocable"* — **no
revocation path reproduced.**

evidence ↗

#4119

Cursor
acme/web

● ADVERSARY · REFUTED

refuted *"no known CVEs"* — sole source was a
vendor blog post.

evidence ↗

#4108

Claude Code
acme/api

● VERIFIER · PASSED

confirmed *"Argon2id params meet policy"* —
reproduced & linked.

evidence ↗

Every agent claim leaves an [audit trail](#) you can open.

Land with one developer's layer → expand to the team's shared brain.

■ APPENDIX · THE PRODUCT · LIVE TEAM MEMORY

One project. One *shared brain*.

Your coworker's agent just made a decision — yours already knows. No duplicated work. No contradictions. The team decides *once*, and every agent respects it.

■ **Product preview** · mockup — not live data

✓ **ships today**: the concurrent file-locked memory merge (flock + atomic-rename, zero lost writes). **roadmap**: the cross-user LIVE sync.

Siloed today across every tool and teammate. re-forge makes it one synced memory you own.

Live team memory

shared context across teammates

project

acme/payments

3 active now

A S D

SHARED CONTEXT · LIVE FEED

S

Shine's agent · 12m ago — **decided**: Argon2id + refresh rotation

evidence ↗

your agent builds on this — won't re-decide it

D

Dana's agent · 4m ago — **found**: Stripe webhook needs an idempotency key

saved to shared memory

⚠

duplicate work prevented — you and Akash both queued "add rate limiter"

merged into one task

SHARED DECISION LOG — every agent on the project respects these

📁

auth = Argon2id + rotation

📁

DB = Postgres, no ORM

📁

no same-day signals in any backtest

✦ re-forge.vercel.app · appendix